常青圈 EvergreenCircle

15分钟生活圈智能体检与规划助手

项目概述

常青圈 **EvergreenCircle** 是基于地图开放能力的社区体检与规划辅助系统，通过真实路网测时、多源 POI 采集与智能诊断算法，为城市社区提供"15分钟生活圈"的量化评估与可视化报告。

> 核心理念：让每个结论都有出处，让每次体检都可复现

参赛赛道：2026 上海开源软件应用创新大赛 · 百度地图命题一

开源许可证：AGPL-3.0

核心能力

🗺️ 等时圈计算（真实路网）

- 渔网采样：中心点 2.5km 范围内分级网格采样（粗扫 400m + 边界加密 150m）
- 批量算路：百度距离矩阵 API（routematrix walking），一次 N×1 并发获取步行耗时
- **IDW** 插值：反距离权重生成连续耗时场，纯 numpy 自实现（不依赖 scipy/shapely）
- 等值线提取：marching-squares 算法提取 5/10/15/20 分钟四级等时圈
- 形态诊断：八方位扇区分析（圆度/最弱方向/未测到顶点披露）

🏪 民生设施统计

- **8** 类民生设施：医疗（医院/诊所/药店）、教育（小学/中学/幼儿园）、购物（菜市场/超市/便利店）、养老（养老院/日间照料）、金融、文体、餐饮、政务
- 三要素硬判：菜市场 / 药店 / 小学 1km 覆盖判定（赛题明确盲区标准）
- 多关键词去重：名称归一化 + 坐标 50m 聚簇，提升查全率

🚨 服务盲区识别

- **1km** 网格扫描：逐格检查三要素覆盖情况
- 连通聚合：相邻盲点聚合成灰区多边形
- 最近设施近邻：标注缺失类别与最近可及点的距离/方向

📊 综合评分模型

- 四维加权：覆盖度(0.40) + 可达性(0.25) + 多样性(0.20) + 均衡性(0.15)
- 盲区扣分：每类盲区扣 4 分，封顶 12 分
- 透明公式：所有参数与权重在报告中完整披露

👥 专家队诊断

- **48** 位虚拟专家：L3 决策层(3人) / L2 策略层(9人) / L1 执行层(36人)
- 按需组队：根据体检需求自动挑选空间定位师/网格规划师/各域顾问
- 章节化报告：医疗/教育/养老/商业逐章诊断，证据溯源可查
- 无 **Key** 降级：LLM 失败时自动退化为规则模板机填，主流程永不阻塞

⚡ 智能体流水线

```
intake → plan → measure → collect → diagnose → report → audit
```

- **SSE** 实时流式：前端 `useTaskStream` 订阅任务进度
- **Trace** 可观测：每个 Agent 的 Prompt/产出/参数/Event 全部落 Trace

📈 双社区对比

- 同图叠加：双圈同比例尺/同中心/叠加三态切换
- 指标差异表：量化设施覆盖差距
- 双雷达图：四维评分直观对比

技术架构

前端技术栈

- 框架：React 19 + TypeScript
- 构建工具：Vite 6
- 状态管理：Zustand
- 地图渲染：BMapGL v3.0（百度 JS API）
- 图表库：ECharts 6 + D3
- **UI** 组件：framer-motion + lucide-react
- 样式：Tailwind CSS

后端技术栈

- 框架：FastAPI 0.115 + Python 3.12
- 数据库：SQLite (WAL 模式)
- 数值计算：numpy (IDW 插值/等值线提取)
- **HTTP** 客户端：aiohttp (并发限流/指数退避)

- 任务编排：自定义多 Agent 引擎（SSE 流式回放）

数据模式（三级兜底）

模式	触发条件	数据来源	标注方式
Live 真实	有百度 AK + 配额充足	百度 API 实时调用	`data_origin='live'`
Cache 历史	同参数 30 天内有记录	SQLite 缓存复用	`served_from='cache'`
Offline 离线	无 AK / 预算耗尽	直线距离 × 绕行系数估算	`data_origin='offline'`
Fixture 演示	演示模式 / 无网络	内置 JSON 快照	`data_origin='fixture'`

前端徽标四态：离线估算 / 历史实时 / 真实数据 / 演示数据，绝不冒充

关键算法

1. 等时圈算法（对应赛题 **40%+30%** 评分）

```
中心点 → 渔网采样(200m间距) → 批量算路(routematrix) →
IDW插值(p=2, k=8) → marching-squares等值线 → GeoJSON输出
```

优化策略：

- 分级采样：粗扫 400m 快扫 → 15min 边界带加密至 150m，总调用量控制在 300 次内
- 批量优先：一次 N×1 距离矩阵替代 N 次单点调用，耗时降低 60-70%
- **IDW** 自实现：纯 numpy 实现，不引 scipy/shapely 重依赖（镜像体积从 800MB 降至 150MB）

实测精度（凯里老街样本）：

- 采样点数：498 点
- 测时成功率：98.2%
- 边界回验误差率：< 5%（抽样 20 个边界内外点二次测时）

2. POI 清洗与去重

```
多关键词检索 → 名称归一化 → 坐标 50m 聚簇 → 类别统计 → IDW耗时回填
```

关键词表（示例：医疗类）：

- 主关键词：`医院`，`诊所`，`卫生院`，`医务室`
- 兜底关键词：`药房`，`药店`，`大药房`
- 过滤规则：排除 `宠物医院`，`兽医` 等非民生类别

去重效果：同一 POI 多关键词命中时，合并为一条记录，查全率提升 23%

3. 盲区识别算法

```
1km网格扫描 → 三要素布尔判定 → 连通分量聚合 → 灰区多边形输出
```

判定标准：

- 硬盲区：1km 圆内同时缺失菜市场 + 药店 + 小学（赛题标准）
- 软盲区：其余 7 类设施中任一类别圈内数为 0

聚合算法：DBSCAN 聚类（eps=200m, min_samples=3），相邻盲点聚合成连续灰区

4. 评分模型

```
总分 =（覆盖度×0.40 + 可达性×0.25 + 多样性×0.20 + 均衡性×0.15）× 100 - 盲区扣分
```

分项定义：

- 覆盖度：8 类设施中圈内有点的类别占比
- 可达性：各类最近设施平均步行耗时（取倒数归一化）
- 多样性：圈内 POI 总数对数归一化
- 均衡性：与双样例基准（凯里/北京劲松）的标准差倒数

盲区扣分：每类硬盲区扣 4 分，封顶 12 分

实测结果

凯里老街 **vs** 北京劲松对比

指标	凯里老街	北京劲松	差异归因
总评分	58.5	56.6	凯里胜在多样性
15min 圈面积	1.36 km²	1.26 km²	凯里路网更稀疏但无障碍
POI 总数	339	356	劲松商业密度更高

指标	凯里老街	北京劲松	差异归因
医疗覆盖	✅ 圈内 3 家	✅ 圈内 5 家	劲松医疗资源更丰富
教育覆盖	✅ 圈内 2 所	✅ 圈内 4 所	劲松学区配套更完善
菜市场监管	❌ 圈内 0 家	❌ 圈内 0 家	两者均为硬盲区
药店覆盖	✅ 圈内 8 家	✅ 圈内 12 家	劲松药店密度更高
小学覆盖	✅ 圈内 1 所	✅ 圈内 2 所	劲松教育资源更密集
盲区总数	3	1	凯里短板更多
等时圈圆度	0.713	0.795	劲松更接近正圆（路网规整）

颠覆性发现：城区成熟 ≠ 步行圈内设施充足。劲松虽为成熟社区，但菜市场和养老设施仍为盲区，需依靠 1km 外资源。

性能基准

场景	采样点数	批量算路耗时	总耗时（含插值）	QPS 消耗
速览档（400m 粗扫）	~170 点	42s	58s	~4 req/s
标准档（分级采样）	~300 点	75s	102s	~4 req/s
精细档（全量加密）	~500 点	125s	168s	~4 req/s

并发优化效果：批量算路相比串行单点调用，耗时降低 **63%**（125s vs 340s）

工程规范

CI/CD 流水线

- **GitHub Actions**（唯一真源）：
  - Backend: pytest (273 项测试)
  - Frontend: lint → typecheck → vitest (228 项) → build
  - API 镜像运行时校验
  - Docker 构建（push-main 时触发）
- **Gitee Go**（镜像子集）：backend + frontend 两个 job

CI 全绿：pytest 273/273 ✅ | vitest 228/228 ✅ | Playwright 21/21 ✅

代码质量

- 后端：Python 3.12 兼容，中文 docstring，密钥环境变量读取
- 前端：TypeScript 严格模式，ESLint 0 error，组件 V 前缀命名
- 测试覆盖率：核心模块 > 85%，契约守卫测试全覆盖

部署支持

- **Docker**：一键 preview（backend + frontend，healthcheck）
- 云端：Vercel / Render / Railway 部署清单齐全
- 自管云：docker-compose + nginx + systemd 完整配置

快速开始

方式 A：一键演示（零配置，推荐评审方）

```
git clone https://github.com/yklg/evergreen-circle.git
cd evergreen-circle
./scripts/run-demo.sh
# 自动打开浏览器 http://127.0.0.1:3400
```

特性：

- ✅ 自动检测 Node.js / Python / 前后端依赖
- ✅ 强制使用内置 fixture 数据（凯里/北京/上海等 5 样例）
- ✅ 禁用所有外部 API 调用（零网络依赖）
- ✅ 自动打开浏览器并打印推荐演示路径

方式 B：**Docker** 一键预览

```
docker compose up --build
# 前端 http://localhost:3400  后端 http://localhost:8010
```

方式 C：本地两命令

```
# 后端
cd backend && python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
python -m uvicorn app.main:app --reload --port 8010
```

```
# 前端
cd frontend && npm install && npm run dev
```

文档体系

文档	内容	位置
README.md	项目说明/快速开始/AK 配置/演示指南	查看
ARCHITECTURE.md	系统架构/两产品域/数据模式/降级链路	查看
AGENTS.md	Agent 体系/专家分层/消息协议/术语表	查看
DEPLOYMENT.md	部署指南/端口真值源/Python 解释器约束	查看
地图**API**调用策略与等时圈算法设计**.md**	调用策略/IDW 插值/实测边界	查看
多源**POI**数据清洗与服务盲区识别算法**.md**	类别表/去重/判盲/评分公式	查看
真实社区对比测试报告**.md**	凯里 vs 北京劲松实跑对比	查看

项目亮点

1. 真实性优先

- 不调用底层路网：完全基于分散点位 API 测时 + 空间插值推导连通区域（赛题鼓励项）
- 实测数据背书：凯里/北京劲松真实 AK 实跑，498 采样点 × 8 类 POI
- 诚实披露局限：离线估算标注「圆形近似」，不冒充真实路网形状

2. 可复现性强

- 口径版本化：四把代次轴（ev/cov/rc/sh）追踪载荷语义变化

- 参数全披露：IDW 幂次/近邻数/绕行系数/容差全部进键
- 契约守卫：B5-B17 系列判据拦截半份发布/口径漂移/值非法

3. 工程化程度高

- 多 **Agent** 编排：48 位专家按需组队，SSE 实时流式展示
- 三级数据源：Live → Cache → Offline/Fixture 优雅降级
- **CI** 全自动化：pytest/vitest/Playwright/Docker 四层防护

4. 用户体验佳

- 零 **Key** 演示：内置 5 份 fixture 快照，评审方开箱即用
- 拖点确认流：拖动中心点仅就地标注，点按钮才触发计算（防误触浪费配额）
- 新旧对比：拖点期间旧等时圈保留半透明作对比

团队与致谢

开发团队：常青圈项目组

AI 协作：本项目深度使用 TRAE 等 AI 编程工具协作完成

技术支持：

- 百度地图开放平台（API 支持）
- 智谱 AI（LLM 网关）
- 开源社区（ECharts/D3/Tailwind 等）

联系方式

- **GitHub**：https://github.com/yklg/evergreen-circle
- 问题反馈：https://github.com/yklg/evergreen-circle/issues
- 邮箱：（待补充）

许可证：AGPL-3.0

最后更新：2026-10-11

版本：v0.7